

GOLICARIE. Geometry Of Lisp Interaction Calculus And Rewriting. Including Extrinsicics.

Goals Achived

We successfully created a proof of concept for a functional programming language with incremental memory usage. This language that can be used to define nodes, graphs, and reduction rules. The graph of nodes will then be reduced until it reaches beta normal form (no more reductions can take place), at which point the resulting graph is printed out. We also created a visualizer for the output graph, as visualizations are much easier for humans to interpret that text.

Node definitions (spiders) are written like this:

```
(spiders (
  (Z
    >r
  )
  (S
    >r
    -n
  )
  (P
    >arg1
    -arg2
    -r
  )
  (T
    >arg1
    -arg2
    -r
  )
))
```

In this example, Z is a node that has an outgoing port.

You can define a graph of nodes:

```
(graph
; Starting off simple. Adding 1 and 2 together.
(
; Successor of zero
S$0.n Z.r
; Is first argument to the addition
S$0.r P$0.arg1
; Successor of the successor of zero
S$1.n Z.r
S$1.r S$2.n
; Is second argument to the addition
S$2.r P$0.arg2
)
)
```

Finally, you can define rules to turn active pairs (two nodes connected by outgoing ports):

```
(rules (
  (
    ; The fact that this won't react when on arg2 is actually perfectly fine.
    ; The goal is to just move everything to the other side
    (Z.r P.arg1)
    ; Because adding to zero is a no-op, we just connect the return to the second
argument
    (P.arg1 P.arg2)
  )
  (
    ; Move from the left to the right
    (S.r P$0.arg1)
    (
      ; We aren't actually "moving things around". The nodes in the active pair
are deleted, but we have a reference to their edges.
      S.r P$0.r
      S.n P$1.r
      P$1.arg1 S.n
      P$1.arg2 P$0.arg2
    )
  )
)
```

Challenges remain

Due to the difficulty of this project, we did not have time to implement the original syntax in the project proposal, instead going with a more pure lisp like language. Additionally, we chose to allow the user to implement a number of core agents such as duplicators, erasers, and superposition, instead of having them be default elements of the language. Thanks to the flexibility of this language, the majority of not yet implemented features could be implemented in the language itself as a library. A core library of nodes and, rewrite rules would make development a lot easier. Additionally, a complete type system would be implemented by defining additional nodes. This language is intended to be a simple wrapper around pure interaction calculus, so we don't want to add numerous of ad hoc features if they are not absolutely necessary. Despite this, there are a number of necessary language features for most projects, such as the definitions for integers, floats, and other primitives.

Improved visual editing abilities would be very useful for a developer. Currently, while we can visualize graphs, there is not a good way to edit them visually.

Additionally, translating this language into a compiled language would allow for significantly improved speed.

The initial goals for evaluating difficult computations quickly did not end up being tested. While the language likely theoretically allows for

Future Works and Road Map

Interaction calculus is important both in its expressiveness and efficiency. Both of these aspects of our language could be greatly improved.

Allowing for “escape hatches” in our language to call a much faster language would allow for both the resource efficiency of the graph structure and the speed of computers at doing low level calculations (addition, multiplication, etc.).

Additionally, the graph structure of interaction calculus allows for fascinating possibilities with type theory. Defining both values and types in the same graph allows for a more expressive (albeit not technically without paradoxes) type system than any mainstream language. For instance, the type of a value could be itself, and operations on values would be equivalent to operations on types (there would be no distinction between type level functions and value level functions).